Decomposition And Reflection Add-On

This is a small addition to the MCP server + RAG system you already built, same repo, same company, same problem. Pick ONE of the two options below and build it.

Option A: Break Down Multi-Part Questions (Query Decomposition)

Real users ask compound questions your search_knowledge_base tool can't answer well in one shot — e.g. “What's our refund policy for late cancellations, and does that change for repeat customers?” Give your server a tool that breaks a question like that into pieces before searching.

- Add one new function/tool decompose_and_search(query, top_k) that sits in front of your existing search_knowledge_base tool — it does not replace it.
- Inside it: make one LLM call that splits the incoming query into 2–4 sub-questions (a numbered list is enough, no dependency graph needed — that's next-level).
- Call your existing search_knowledge_base once per sub-question, and tag each returned chunk with which sub-question it came from.
- Return the combined, tagged chunks as one result — don't try to merge them into a single answer yourself, let the model do that part.
- Show one demo query that your normal search tool answers only partially, and that decompose_and_search answers fully.

Option B: Add a Grounding Check Before You Trust an Answer

Give your agent one guardrail: before it hands back an answer built from retrieved chunks, make it check whether the answer is actually supported by what was retrieved — and retry once, and only once, if it isn't.

- Write generate_answer(query) the normal way: search → build a draft answer from the top chunks.
- Add one critique step: ask the model “is every claim in this draft backed by the chunks below? Answer PASS or FAIL and why.”
- On FAIL, do exactly one retry: rewrite the search query based on the critique, search again, generate a second draft. Hard-cap it at one retry — no loop.
- If the retry still fails, have the agent say it couldn't find a grounded answer instead of guessing.
- Show one demo query where the first draft fails the check and the retry either fixes it or honestly reports it can't.

Starter code: two Python files are provided alongside this handout decompose_search.py for Option A and grounded_reflect.py for Option B. Each runs standalone with fake stand-ins for your LLM client and search tool so you can see the flow before wiring in your real ones (look for the TODO comments).